UNIVERSITY OF SALERNO

Department of Computer Science

Master Degree in Internet of Things

GRADUATION THESIS

Evaluating automated methods for performance microbenchmark generation

SUPERVISORS

Prof. Dario Di Nucci

Dott. Antonio Trovato

University of Salerno

CANDIDATE

Ulugbek Abdimurodov

Matricola: 0522501394

2025-2026 Academic Year

This thesis was carried out in the

Talent without working hard is nothing

Cristiano Ronaldo

Abstract

The accurate evaluation of software performance is a critical aspect of modern software engineering, where efficiency requirements, such as execution speed and resource usage, play a pivotal role. Microbenchmarks are essential for assessing these non-functional requirements, yet their manual creation is time-consuming and prone to errors. Automated methods offer a promising alternative, but they also introduce challenges, such as ensuring accuracy, mitigating execution variability across environments, addressing scalability issues, and managing dependency on predefined test cases in traditional performance testing frameworks. This study explores and evaluates automated methods for generating performance microbenchmarks, focusing on two distinct approaches: ju2jmh, an automation tool that converts unit tests into microbenchmarks, and Large Language Models (LLMs), such as GPT-4, which directly generates microbenchmarks from source code.

The study aims to rigorously compare these two paradigms by running ju2jmh and LLM on different versions of Apache Ignite. We validate both on Apache Ignite using pairs of commits with documented performance regressions, providing clear ground truth for whether generated benchmarks distinguish pre-fix from post-fix behavior. The findings are expected to shed light on the advantages and disadvantages of using traditional automation tools versus modern AI-based solutions to generate performance microbenchmarks. In our study, the LLM approach performed best overall, detecting more regressions.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Application Context

Performance testing is a crucial practice in software engineering used to determine how well an application functions under varying conditions and workloads. It evaluates the non-functional attributes of a software system, specifically focusing on metrics such as execution duration, response time, resource usage (like CPU and memory), and overall system stability.

The primary objective of performance testing is to uncover efficiency issues and performance bugs before the software is released to users. While these types of bugs might not break the core functionality of the application, they can severely degrade the user experience by making the system slower or causing it to consume excessive resources. By executing performance tests, developers can ensure that their applications are fast, reliable, and scalable enough to handle numerous concurrent users [1, 2].

Typically, performance testing is conducted by repeatedly running the application

under test to collect a statistically significant number of performance data points. This evaluation can be performed at different levels of granularity, ranging from system-level testing that evaluates the entire application in production-like environments, to unit-level tests that assess the execution efficiency of small, isolated code segments [3, 4]

The primary methodology used to conduct this evaluation is benchmarking, which measures software performance to establish baselines and compare results over time. This continuous comparison helps development teams detect performance bugs—issues that may not break the software's functionality but severely degrade the user experience by making it slower or more resource-intensive. Benchmarking strategies generally fall into two main categories: macrobenchmarks (application benchmarks) and microbenchmarks [4, 5].

Macrobenchmarks are comprehensive tests designed to evaluate the overall performance of an entire system by simulating real-world workloads in production-like environments. While they provide a highly realistic view of how a system will behave under actual user load, they are notoriously complex to set up and configure. Furthermore, application benchmarks are resource-intensive and computationally expensive, often taking several hours to execute. This makes them impractical to run frequently within fast-paced Continuous Integration and Continuous Deployment (CI/CD) pipelines.

To address the limitations of large-scale system testing, developers increasingly rely on microbenchmarking. Microbenchmarks are small, highly isolated test cases designed to evaluate the performance of specific, fine-grained code segments, such as individual methods, algorithms, or even single statements. Unlike macrobenchmarks that evaluate end-to-end system behavior, microbenchmarks operate at a unit-level granularity, functioning similarly to functional unit tests but focusing strictly on execution efficiency[4, 3, 5].

However, extracting accurate data from microbenchmarks requires rigorous testing methodology. In execution environments like the Java Virtual Machine (JVM), code undergoes an initial "warm-up" phase where dynamic optimizations like Just-

In-Time (JIT) compilation and memory mapping occur. To obtain reliable and representative measurements, microbenchmarks must be repeatedly executed during this warm-up phase until the system reaches a stable "steady state" of performance before actual data collection begins [2, 6, 1]

1.2 Motivation and Objectives

Despite the critical importance of software efficiency, developing and maintaining dedicated performance test suites remains a significant challenge in modern software engineering. Creating effective microbenchmarks is a notoriously complex and labor-intensive task. It requires developers to possess deep expertise in the underlying execution environment. For instance, testing on the Java Virtual Machine (JVM) requires careful management of dynamic optimizations—such as Just-In-Time (JIT) compilation, garbage collection, and proper warm-up phases. If these factors are not properly handled, execution time measurements can be easily distorted, leading to inaccurate conclusions [2].

Because of this high barrier to entry, rigorous performance microbenchmarking is far less common than functional unit testing. Consequently, many software projects lack dedicated performance test suites entirely. Developers are often forced to rely on unreliable, ad-hoc methods to measure efficiency, such as running standard JUnit tests in a loop, which fails to provide the necessary warm-up and isolation mechanisms required for stable measurements.

This scarcity of reliable performance tests provides a strong justification for investigating automated benchmark generation. By automatically deriving performance microbenchmarks, we can significantly democratize performance testing and reduce the manual burden on developers. Furthermore, automation ensures that the generated tests adhere to the strict methodological standards of specialized frameworks, such as the Java Microbenchmark Harness (JMH), seamlessly handling the complex execution scaffolding required for reliable results [4].

To address this need, this thesis investigates automated methods for generating

JMH microbenchmarks to track performance changes across software versions. Specifically, this research focuses on evaluating and comparing two distinct generation strategies: the ju2jmh-augmented LLM approach and the LLM Only approach

1. **Ju2jmh-Augmented LLM:** This methodology implements a multi-stage hybrid pipeline designed to combine the generative flexibility of LLM with the deterministic reliability of the ju2jmh framework. In this approach, the LLM is first utilized to synthesize JUnit test cases that specifically exercise the code segments or changed methods under investigation. These AI-generated functional tests then serve as the primary input for the ju2jmh tool, which performs the structural transpilation into formal Java Microbenchmark Harness (JMH) benchmarks. This ensures that the benchmarks are tailored to the specific code logic while the subsequent rule-based conversion by ju2jmh guarantees that the final benchmarks adhere to the rigorous boilerplate, annotation, and configuration requirements essential for stable performance testing

2. **LLM Only:** This method explores the potential of Large Language Models to generate JMH benchmarks directly from source code, bypassing intermediate frameworks. LLMs have demonstrated significant capability in code generation and repair tasks. This approach evaluates whether the GPT-4.1 model can produce more idiomatic benchmarks or handle edge cases (such as complex setups or teardowns) more effectively than template-based tools. We aim to determine whether LLMs have reached a level of proficiency that enables them to understand the semantic requirements of performance testing without rigid structural guidance.

Ultimately, this research seeks to determine whether modern LLMs perform better when guided by traditional analysis tools and unit tests, or can natively generate superior performance benchmarks directly from source code.

1.3 Results Obtained

We have chosen to study these two specific methods to explore the fundamental trade-off between strict structural guidance and pure generative flexibility. By run-

ning benchmarks generated by both methods under identical JMH configurations and the same environmental settings, and statistically comparing execution times before and after code fixes, this thesis aims to determine the most effective strategy for automating performance testing.

The research conducted in this thesis led to the successful implementation and evaluation of an automated pipeline to generate Java Microbenchmark Harness (JMH) tests. By targeting specific Java methods across different software versions—specifically, comparing a parent commit with a bug-fixing commit—the project established a systematic environment to track execution times and identify performance changes.

The generation of these microbenchmarks was achieved by exploring and comparing two distinct strategies—the hybrid ju2jmh-augmented LLM approach and the standalone LLM approach—utilizing GPT-4 on the Apache Ignite repository. To ensure an objective evaluation, we selected targets based on a previous mining study by Campos et al., which identified documented performance regressions within these repositories. For each of the three analyzed targets in Ignite, we selected a specific pair of commits: a pre-fix commit, representing the software version in which the performance issue was present, and a post-fix commit, representing the version in which the issue was resolved. To rigorously evaluate the effectiveness of the generated tests, the execution data was analyzed through a robust hierarchical bootstrap statistical analysis. This advanced method is crucial for performance testing because it accurately accounts for the complex two-level variation inherent in JMH executions—both the variation between different JVM instances (forks) and the variation across iterations within the same fork—thereby providing high confidence in distinguishing actual performance shifts from random environmental noise.

The experimental evaluation revealed distinct differences in the efficacy of the two generation methods. While the tool-augmented pipeline provided strong structural guarantees by anchoring the benchmarks in functional unit tests, it exhibited limitations in capturing the actual performance regressions. In contrast, the purely generative LLM approach demonstrated a noticeably higher sensitivity and effectiveness in successfully detecting performance differences. Unconstrained by the rigid structure of a deterministic translation tool, LLM-generated benchmarks are better

equipped to target the performance-critical code paths.

Ultimately, the findings of this study confirm that automating the generation of performance test suites is a viable and valuable strategy to address the widespread scarcity of microbenchmarks in software projects. Furthermore, the results highlight the significant potential of relying on the generative flexibility of modern Large Language Models to create meaningful, robust performance tests capable of reliably alerting developers to efficiency degradations.

1.4 Thesis Structure

The remainder of this thesis is structured into the following chapters:

- **Chapter 2 - Background and Related Work:** In this chapter, fundamental concepts are described and already existent approaches are presented, emphasizing how this thesis differs from them with a new provided contribution;
- **Chapter 3 - Ju2JMH-augmented LLM and LLM:** This chapter provides detailed information on how I prepared the system, how I used Prompt engineering in JMH, what methods I used;
- **Chapter 4 - Data Analysis and Results** This chapter describes the results I obtained during the project, and how I compared them;
- **Chapter 5 - Threats to Validity:** This chapter describes the possible threats to the validity of the assessments conducted on the designed system;
- **Chapter 6 - Conclusions:** This chapter describes a brief of all the thesis is presented and some future works are discussed

CHAPTER 2

Background and Related Work

In this chapter, fundamental concepts are described and already existent approaches are presented, emphasizing how this thesis differs from them with a new provided contribution.

2.1 Performance Testing

Ensuring modern software reliability extends beyond verifying functional correctness; it necessitates evaluating non-functional requirements. Performance testing is the practice of systematically assessing key system attributes, including execution duration, response latency, overall stability, and resource utilization under varying conditions and workloads. The primary objective is to identify bottlenecks and uncover efficiency issues before the software reaches end users [1, 2, 7].

Unlike functional defects, which typically cause immediate application crashes, performance issues tend to accumulate gradually. These degradations severely compromise responsiveness and scalability, leading to a poor user experience, wasted computational resources, and significant financial consequences [2, 7, 8, 9, 10].

Usually, system performance is evaluated by repeatedly executing the application to collect a statistically significant number of data points. However, traditional system-level performance testing is notoriously challenging [1, 3]. It is an expensive, resource-intensive, and time-consuming process requiring the simulation of realistic, production-like workloads. Because setting up these environments is complex, system-wide testing is frequently conducted late in the release cycle, making it difficult to pinpoint the exact code changes responsible for a regression [5, 10, 11].

2.1.1 Microbenchmarking

Benchmarking is a fundamental methodology utilized during performance testing to systematically measure how a software application functions and to compare its results over time. The primary objective is to establish precise performance baselines and uncover detrimental efficiency bugs—such as slow response times or excessive resource utilization—before a new software version reaches end-users [2, 12, 4, 5]. Traditionally, this evaluation is achieved through application benchmarks, frequently referred to as macrobenchmarks. These tests deploy an entire software system in a production-like environment and stress its external interfaces with simulated user workloads [3, 11, 5]. While macrobenchmarks undoubtedly provide a realistic view of end-to-end system behavior, they are notoriously complex to configure, computationally expensive, and frequently require hours or even days to completely execute [13].

To directly overcome the heavy execution costs and slow feedback loops associated with full-system testing, developers increasingly rely on a highly granular technique known as **microbenchmarking** [2, 3, 4, 14]. Rather than evaluating an entire application from an external black-box perspective, microbenchmarks assess the execution efficiency of extremely small units of source code, such as individual methods, data structures, or specific algorithms [2, 4, 7, 11]. This strategy operates at a deep white-box level and functions very similarly to traditional functional unit tests [15, 5]. However, instead of simply checking if a test passes or fails based on logical correctness, microbenchmarks strictly extract precise performance metrics,

most notably the average execution time, latency, and overall throughput [4, 15].

The distinct advantages of microbenchmarking are exceptionally substantial. Because they evaluate tiny fragments of code, microbenchmarks inherently offer remarkably short execution durations [3]. This lightweight nature makes them well-suited for seamless integration into Continuous Integration and Continuous Deployment (CI/CD) pipelines. By executing automated microbenchmark suites on every new code commit, development teams receive immediate performance feedback, allowing them to pinpoint and resolve fine-grained performance regressions at the exact subroutine where they were originally introduced [11, 13].

Despite these compelling benefits, extracting reproducible measurements is highly challenging in managed runtime environments like the Java Virtual Machine (JVM). The JVM actively employs dynamic features such as Just-In-Time (JIT) compilation and aggressive profile-guided optimizations, which collectively cause initial execution times to fluctuate wildly. To obtain reliable data, the targeted code must undergo a rigorous "warm-up" phase of repeated executions until the underlying environment reaches a steady-state of execution. To successfully navigate these environmental complexities, developers must utilize specialized testing frameworks, such as the Java Microbenchmark Harness (JMH), to systematically orchestrate executions, isolate code segments, and ultimately achieve accurate **microbenchmarking** [2, 9, 16].

2.1.2 Java Microbenchmark Harness

Java Microbenchmark Harness (JMH) was developed under the OpenJDK umbrella. Today, JMH has emerged as the de facto standard framework for constructing, executing, and analyzing microbenchmarks for Java and other JVM-based languages. By providing a structured, annotation-based infrastructure that heavily resembles JUnit, JMH successfully abstracts away the underlying execution scaffolding, allowing developers to focus strictly on the code being measured [2, 15, 9].

When configuring a JMH microbenchmark, developers must explicitly define the **Benchmark Mode**, which dictates the specific performance metric the framework will collect during execution. The natively supported modes include:

- Throughput: Measures the number of operations executed per unit of time (e.g., operations per second).
- Average Time: Calculates the average execution time required to complete a single operation.
- Sample Time: Extracts a statistical distribution of execution times, which is highly useful for identifying performance spikes and tail latencies.
- Single Shot Time: Measures the duration of a single, cold-start invocation without any prior JVM warmup.

Measurements and TimeUnit. Coupled with the execution mode, JMH allows developers to define the desired output *TimeUnit*. There are different types of TimeUnites for different cases in Table **??**. This ensures the reported metrics are scaled appropriately based on the targeted function's execution duration.

Measurements and TimeUnit Coupled with the execution mode, JMH allows developers to define the desired output *TimeUnit*. The available *TimeUnit* values and their ideal use cases are summarized in Table 2.1. This ensures the reported metrics are scaled appropriately based on the targeted function's execution duration.

Table 2.1: JMH TimeUnit Types and ideal cases

TimeUnit Type	**Ideal Use Case**
Nanoseconds	Extremely fast, low-level operations (e.g., bitwise shifts, basic math).
Microseconds	Standard method calls and lightweight data structure operations.
Milliseconds	I/O-bound tasks or complex algorithmic processing.
Seconds	Measuring overall throughput and transaction rates.

To accurately interpret the results of a microbenchmark, one must understand the fundamental units of measurement used by the harness. JMH structures its execution around three nested levels: operations, iterations, and forks.

The most granular unit is the Operation, which represents a single execution of the target code, method, or algorithm under analysis. Because a single operation often executes in nanoseconds or microseconds—intervals too small to be timed reliably due to system clock precision—JMH groups them into Iterations.

An Iteration is a fixed period of time (typically one second) during which the harness executes the target method as many times as possible. The primary metric reported is usually the average time per operation or the number of operations per unit of time (throughput) calculated across the entire iteration.

JMH orchestrates a rigorous execution lifecycle to guarantee statistically sound measurements and prevent misleading results.

Initial Java executions are highly unstable due to Just-In-Time (JIT) compilation and class loading. JMH allows developers to configure warmup iterations, where the target code runs repeatedly to allow the JVM to reach a "steady state." Crucially, the performance data from these iterations is discarded and does not contribute to the final results [17].

Once the warmup phase is complete and the code is optimized, the harness enters the measurement phase. The data collected during these iterations is recorded and used to calculate the final statistical distributions, medians, and confidence intervals used in the analysis.

- **Fork**: To guarantee complete execution isolation and prevent cross-contamination between tests, JMH utilizes forks. A fork spawns a completely new, isolated JVM instance for the benchmark execution. This prevents profile-guided optimizations generated during one test from bleeding into and distorting the measurements of another.
- **State and Threads:** The *@State* annotation manages state information and defines the scope of instantiated objects, dictating whether they are shared across all testing threads or kept strictly thread-local. Furthermore, JMH natively supports multi-threaded executions to assess concurrent code efficiency.

- **Setup, TearDown, and Levels:** JMH utilizes *@Setup* and *@TearDown* fixture annotations to initialize testing environments and clean up resources outside the measurement window. Crucially, they can be configured at different **Levels**: *Trial* (executes once per fork), *Iteration* (executes before/after each iteration), and *Invocation* (executes before/after each individual method call, though the latter is heavily discouraged for short-running tasks due to the massive JMH overhead it introduces) [9].
- **Parameters:** The *@Param* annotation allows developers to define various input workloads. JMH automatically generates the Cartesian product of all parameters and executes the benchmark for each combination [15].
- **Blackhole:** The JVM aggressively applies dead-code elimination, removing code whose return values are never used. To prevent this, JMH provides the Blackhole facility, an object designed specifically to consume method return values and force the JVM to execute the underlying logic [9].

Figure 2.1: Benchmark Example

```java
import org.openjdk.jmh.annotations.*;
import org.openjdk.jmh.infra.Blackhole;
import java.util.concurrent.TimeUnit;
import java.util.ArrayList;
import java.util.List;

@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1, timeUnit = TimeUnit.SECONDS)
@Measurement(iterations = 10, time = 1, timeUnit = TimeUnit.SECONDS)
@Fork(1)
@State(Scope.Thread)
public class ListBenchmark {

    @Param({"100", "1000", "10000"})
    private int size;

    private List<Integer> list;

    @Setup(Level.Trial)
    public void setup() {
        list = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            list.add(i);
        }
    }

```

```
    @TearDown(Level.Trial)
    public void teardown() {
        list.clear();
    }

    @Benchmark
    public void testListAccess(Blackhole bh) {
        for (int i = 0; i < size; i++) {
            // The Blackhole consumes the value to prevent dead-code elimination
            bh.consume(list.get(i));
        }
    }
}
```

A practical example (Figure 2.1)demonstrating the structure and essential annotations of a JMH microbenchmark class. This fundamental class structure completely encapsulates JMH's power: establishing a controlled state, isolating variables through parameters, warming up the JVM, and accurately measuring execution time without falling victim to runtime optimizations.

2.2 Large Language Models (LLMs)

Large Language Models (LLMs) represent a significant breakthrough in artificial intelligence and modern software engineering. Historically, the application of deep learning to language tasks relied heavily on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These sequential models served as the primary foundation for early Neural Machine Translation systems. However, the landscape dramatically shifted with the introduction of the Transformer architecture. By relying on self-attention mechanisms rather than strictly sequential processing, Transformers enabled the training of models on unprecedented amounts of natural language text and public source code. Through this extensive pre-training, modern LLMs accumulate vast linguistic and structural knowledge, exhibiting exceptional reasoning and generative capabilities [18].

LLMs can be broadly categorized into different architectural types based on their underlying mechanisms. First, bidirectional models, such as BERT and its code-specific variant CodeBERT, excel at understanding the context of a sequence from both direc-

tions, making them highly effective for code representation and classification tasks. Second, sequence-to-sequence (encoder-decoder) models, such as T5 and CodeT5, map an input sequence to a distinct output sequence, which is particularly useful for translation-like tasks. Finally, generative transformer models (decoder-only), such as the GPT series, are explicitly trained to predict the next token most likely to follow a given input. Beyond architecture, LLMs are practically divided into open-source and closed-source models [7, 18, 14].

The primary advantage of utilizing LLMs in software engineering is their deep, statistical understanding of programming semantics and syntax across multiple paradigms. This allows them to produce natural-looking code that closely resembles human-written logic. Unlike traditional static analysis or rule-based tools, LLMs possess a high degree of generative flexibility and do not rely on rigid heuristics. Furthermore, they significantly reduce the cognitive demand required to understand complex codebases by providing context-aware code generation and semantic analysis [7].

LLMs should be leveraged when tackling complex, labor-intensive tasks that benefit from semantic pattern recognition. Today, LLMs are highly effective for automated program repair (translating buggy code snippets into fixed code), performance mutation testing (generating controlled transformations that intentionally degrade performance to simulate real-world efficiency bugs), and automated test generation (synthesizing comprehensive unit tests and performance microbenchmarks complete with appropriate setup configurations) [7, 18, 19].

To successfully harness the powerful code generation capabilities of these large models, developers employ two primary methodologies. For open-source models, developers use fine-tuning, adjusting the model on specific datasets—such as thousands of historical bug-fix pairs—to optimize it for downstream tasks. For closed-source models, developers utilize prompt engineering, carefully structuring the input instructions, context, and data to effectively guide the model's output [18].

2.2.1 Prompt Engineering

Prompt engineering has emerged as an indispensable discipline when interacting with modern Large Language Models, particularly for closed-source models accessed via APIs where traditional weight fine-tuning is impossible. It is the systematic practice of carefully crafting, structuring, and optimizing the natural language inputs—known as prompts—to effectively guide the generative capabilities of the model toward a highly specific, accurate output. In complex software engineering tasks, such as automated test generation or performance mutation testing, a poorly designed prompt often leads to invalid syntax, hallucinatory methods, or code that completely fails to capture the intended benchmarking logic [20].

A highly effective prompt is rarely a simple question; rather, it is a rigorously structured template composed of several distinct elements. Recent research guidelines suggest dividing prompts into four primary components: Instructions, Context, Input Data, and Output Indicators. The Instruction explicitly directs the LLM on what task to perform, such as generating a Java Microbenchmark Harness (JMH) test or mutating a specific algorithm. The Context provides the necessary background, clarifying constraints and domain-specific rules. The Input Data supplies the exact target elements, such as the specific method signature or source code to be processed. Finally, the Output Indicator strictly defines the required format of the model's response, ensuring the generated code or structured JSON can be seamlessly parsed by automated execution pipelines [7].

A critical challenge in prompt engineering for software tasks is managing the volume of code provided to the model. While it is tempting to supply an entire class file to ensure the LLM has all possible information, this "full context" approach often exceeds token limits, wastes computational resources, and causes semantic overload by confusing the model with irrelevant details. To resolve this, advanced prompting strategies employ Context Condensation. Instead of dumping raw files, the prompt is enriched only with surgically extracted information: the focal method's signature, the method's direct body, essential class dependencies, invoked method signatures, and critical documentation comments. Furthermore, including real-world usage examples mined from software documentation significantly anchors the LLM's understanding,

allowing it to generate benchmarks that invoke APIs with sensible, realistic values rather than random, meaningless inputs. By filtering out redundant code, the LLM maintains strict focus on the target behavior [19].

Beyond static structuring, prompt engineers leverage advanced paradigms to boost reasoning and accuracy. Few-shot learning involves embedding concrete examples of inputs and desired outputs directly within the context window. For instance, when asking a model to generate performance mutants or fix a JMH bad practice, providing a few pairs of real-world examples alongside their explanations drastically improves the relevance of the generated results. Furthermore, techniques like Chain-of-Thought (COT) prompting explicitly instruct the model to articulate its step-by-step reasoning before outputting the final code. By forcing the model to conceptually break down the benchmarking constraints before writing the syntax, the logical correctness of the generated artifact substantially increases [7].

Finally, prompt engineering in software generation is rarely a single-shot endeavor. Modern LLM-based testing frameworks heavily rely on an adaptive, iterative prompting mechanism, often referred to as a Generation-Validation-Repair loop [19]. Because initial completions may contain compilation errors or fail at runtime, the prompt pipeline must dynamically adjust. If a generated test fails, an automated validator extracts the exact error message and the failing code [21]. A Repair Prompt is then systematically constructed, supplying the LLM with the faulty JMH test, the specific compiler or assertion error, and an explicit instruction to fix the identified issue. This iterative reprompting allows the LLM to leverage its deep semantic understanding to dynamically refine and debug its own output, substantially elevating the final success rate of complex code generation tasks.

2.3 JU2JMH

The Gap Between Functional and Performance Testing In modern software engineering, functional unit testing is a deeply established practice, with developers routinely writing comprehensive test suites using frameworks like JUnit to ensure logical correctness. However, performance microbenchmarking is far less common, largely due to the steep learning curve and significant manual effort required to con-

struct reliable tests. Writing Java Microbenchmark Harness (JMH) tests necessitates a profound understanding of the Java Virtual Machine (JVM), dynamic Just-In-Time (JIT) compilation, and complex framework configurations. Because of these barriers, many software projects lack dedicated microbenchmark suites entirely, and developers often resort to the unreliable ad-hoc practice of executing standard JUnit tests in a loop to measure execution time. This practice is highly vulnerable to environmental noise and completely lacks the rigorous warm-up iterations and execution isolation provided by specialized frameworks like JMH [4, 22].

To bridge this substantial gap between functional and performance testing, researchers introduced *ju2jmh* [4], an automated framework designed to systematically convert existing JUnit test suites into ready-to-execute JMH microbenchmarks. By leveraging the widespread availability of JUnit tests, *ju2jmh* relieves developers from the intense manual labor and domain expertise required to build microbenchmarks from scratch, simultaneously providing the rigorous execution scaffolding inherent to JMH[4].

The Architecture and Internal Workflow of ju2jmh. The operation of ju2jmh is highly deterministic and relies on advanced static analysis and code generation techniques. The framework's core workflow is divided into two primary phases: the Analysis step and the Benchmark Generation step[4].

In the Analysis step, ju2jmh statically inspects the target project to identify unit test classes and extract their execution lifecycle features. To achieve this, it utilizes the Apache Commons BCEL (Byte Code Engineering Library) to perform deep bytecode analysis. The tool scans the bytecode for standard JUnit annotations to precisely locate test methods (annotated with @Test). Crucially, it does not stop at the test methods themselves; it also identifies fixture methods (annotated with @Before, @After, @BeforeClass, or @AfterClass), test rules (annotated with @Rule or @ClassRule), and expected exceptions defined in the test annotations. This thorough extraction ensures that the generated performance benchmark will perfectly replicate the environmental setup and teardown phases originally intended for the functional test.

Following the analysis, the framework proceeds to the Benchmark Generation step, where it utilizes the JavaParser library to manipulate the source code at a structural

level by generating and modifying Abstract Syntax Trees (ASTs). Instead of directly modifying the original test files or creating entirely disconnected classes, ju2jmh creates an exact copy of the AST of the relevant JUnit test class. Within this copied AST, it inserts a newly generated JMH benchmark class—typically named `_Benchmark`, as a static member class. This static member class contains the JMH benchmark methods (annotated with @Benchmark), each specifically responsible for repeatedly executing a copy of the original JUnit test method as its payload [4].

Structural Design and Execution Management to ensure the correct execution of the JUnit lifecycle within the JMH framework, ju2jmh employs a specialized inheritance hierarchy. Each generated benchmark class inherits from a common superclass designed specifically to bridge the two frameworks. This superclass implements all the necessary functionality to instantiate the JUnit test class, invoke its extracted fixture methods, apply its test rules, and handle expected exceptions during the benchmark execution.

The architectural decision to use copies of the original unit tests as the benchmark payload, rather than directly referencing the original classes, provides significant flexibility. This design isolation allows ju2jmh to potentially modify the benchmark payloads in the future—such as injecting JMH Blackhole objects to explicitly prevent aggressive dead-code elimination by the JVM—without polluting or breaking the original functional test suite. Furthermore, placing the generated benchmark class as a static member of the copied test file avoids naming conflicts and logically organizes the performance tests alongside the functional logic they evaluate [4].

In the Empirical Effectiveness and Evaluation, Extensive empirical evaluations utilizing Performance Mutation Testing (PMT) have demonstrated that ju2jmh is highly effective. When compared to the ad-hoc practice of running standard JUnit tests, the benchmarks generated by ju2jmh exhibit significantly greater stability, characterized by a much lower Relative Standard Deviation (RSD) across repeated executions. By utilizing JMH's warm-up iterations, ju2jmh effectively mitigates the JIT compilation noise that plagues raw JUnit executions.

Furthermore, ju2jmh benchmarks excel in bug detection. In experiments where artificial performance bugs (mutants) were injected into the source code, ju2jmh consistently achieved higher mutation scores than standard JUnit tests, proving more

capable of detecting performance degradations. When compared to manually written JMH benchmarks crafted by expert developers, ju2jmh performs comparably in terms of stability and sensitivity. However, ju2jmh holds a massive advantage in codebase coverage; while manually written benchmarks only target highly specific, narrow code paths, ju2jmh automatically scales to cover the entire unit test suite, executing a far wider variety of code and killing a much larger total volume of mutants. It also vastly outperforms alternative automated tools like AutoJMH, which is restricted to generating benchmarks for single, manually specified statements and is currently obsolete.

Limitations of the Framework. Despite its robust mechanical translation, the strictly deterministic nature of ju2jmh introduces certain limitations. First, its effectiveness is strictly bound by the quality and coverage of the existing JUnit test suite; if a project lacks comprehensive functional tests, ju2jmh cannot generate meaningful performance benchmarks. Secondly, ju2jmh struggles with parameter optimization. It relies heavily on JMH's default parameters (e.g., default iteration counts and warm-up times) and does not intelligently analyze the specific characteristics of the target code to automatically determine optimal configuration parameters. Finally, because it blindly converts functional logic into performance logic, it suffers from inadequate contextual analysis; unit tests are often not designed with performance benchmarking in mind, and simply looping them can sometimes yield unrealistic execution states or invoke heavy mock dependencies that distort actual system performance. These specific limitations highlight the boundaries of purely rule-based static translation and motivate the exploration of generative AI and Large Language Models (LLMs) to construct more context-aware, idiomatic microbenchmarks.

2.4 Related Work

The pursuit of automated performance evaluation has evolved from static, rule-based transformations to dynamic, intelligence-driven generation. While significant research exists in the fields of automated testing and generative AI, this thesis occupies a unique niche. The landscape of performance engineering and automated software testing has seen significant advancements in recent years. This section sur-

veys the existing literature across four key research pillars: automated benchmark generation, Large Language Models (LLMs) in software engineering, performance regression testing in distributed systems, and comparative methodologies. While prior research has investigated these areas independently, there is a distinct gap regarding the direct comparison of deterministic, rule-based translation tools against generative artificial intelligence for detecting performance regressions in complex, large-scale systems like Apache Ignite.

Automated Benchmark Generation. Traditional methods for creating performance benchmarks have historically relied on static analysis, dynamic analysis, and search-based software engineering (SBSE). For example, Rodriguez-Cancio [16, 23] developed AutoJMH, a tool that utilizes static analysis to automatically extract single statements and loops into microbenchmarks designed to prevent aggressive JVM optimizations like dead-code elimination and constant folding. Building upon this need for automation, Jangali developed ju2jmh, a framework that leverages abstract syntax tree (AST) manipulation to deterministically translate existing functional JUnit tests into fully scaffolded JMH microbenchmark [23].

However, their approaches were limited to rigid, rule-based structural translations and failed to account for complex state management and parameter optimization. Because these tools rely strictly on the existing logic of functional tests, they suffer from inadequate contextual analysis; they blindly loop functional logic that was often not designed for performance evaluation, which can yield unrealistic execution states and distort measurements. Furthermore, early tools like AutoJMH are highly restricted to manually specified single statements, drastically limiting their scalability. In contrast, this thesis addresses this by utilizing LLMs to capture the context of the target methods. By moving beyond rigid static analysis, we investigate whether generative AI can bypass the constraints of existing functional tests to synthesize more idiomatic, performance-focused microbenchmarks natively [7][8].

LLMs for Code Synthesis and Testing. The application of Large Language Models for Software Engineering (LLM4SE) is currently a rapidly expanding domain. Schäfer developed TestPilot, an adaptive system that successfully utilizes off-the-shelf LLMs, such as GPT-3.5, to automatically generate high-coverage JavaScript unit tests by prompting the model with method signatures and documentation. Similarly, various

studies have demonstrated the efficacy of fine-tuned LLMs and few-shot prompting in generating complex test oracles and performing automated program repair [19, 18]. Their approaches were limited to evaluating functional correctness and failed to account for the profound complexities of performance testing in managed environments. Generating functional unit tests is well-researched, but synthesizing performance microbenchmarks is significantly harder because it requires a deep, structural understanding of JVM warm-up phases, Just-In-Time (JIT) compilation, memory management, and environmental noise mitigation [2, 24]. Without this domain-specific understanding, generated code frequently falls victim to JMH bad practices, such as zero-fork configurations or ignored return values [9].

This thesis addresses this by utilizing LLMs to explicitly generate JMH-compliant performance microbenchmarks, rather than just functional unit tests. We explore whether modern LLMs (like GPT-4) possess the inherent semantic knowledge to correctly configure complex JVM benchmarking scaffolding.

Performance Regression Testing in Distributed Systems. Detecting performance regressions—where code changes gradually degrade efficiency—is notoriously difficult, particularly in large-scale distributed architectures. Ding et al. [25]analyzed performance issues in distributed data processing systems like Hadoop and Cassandra, exploring whether readily available functional tests from release pipelines could serve as proxies for performance tests. Furthermore, Grambow et al. [5] extensively studied application-level benchmarking to track performance changes across multiple software versions in cloud-based time-series databases.

However, their approaches were limited to either repurposing functional tests (which often had too light of a workload to reliably trigger performance anomalies [25]) or executing full-system application macrobenchmarks. Macrobenchmarks take a black-box perspective and are notoriously expensive, resource-intensive, and time-consuming to execute, making it extremely difficult to pinpoint the exact code-level commit responsible for a regression [5].

In contrast, this thesis addresses this by exploring automated micro-level detection specifically tailored to Apache Ignite. By utilizing microbenchmarks to isolate specific methods before and after bug-fixing commits, our approach facilitates a lightweight, white-box strategy to identify precise regressions without the heavy overhead of

deploying an entire distributed cluster.

Comparative Studies: Rules vs. AI Summary: As AI capabilities mature, researchers have begun to compare traditional static tooling against generative AI. Zhi et al. [26] recently proposed the ChatPerfTest framework, highlighting the shortcomings of rule-based tools like ju2jmh, and proposing a specialized LLM pipeline with context condensation to generate and repair JMH microbenchmarks. Additionally, Zhong et al. [18] conducted comprehensive evaluations of various Neural Program Repair systems, mapping their effectiveness against traditional search-based software engineering techniques.

However, their comparative studies were limited in scope and failed to account for direct, in-the-wild effectiveness against real performance regressions. There remains a distinct lack of empirical research directly comparing a dedicated, deterministic rule-based tool (ju2jmh) against a purely generative AI approach specifically for the task of isolating and detecting actual performance regressions across software versions.

In contrast, this thesis addresses this by executing a rigorous comparative study on a complex distributed project (Apache Ignite). By utilizing a hierarchical bootstrap statistical analysis to evaluate the execution times of parent and fixing commits, we definitively quantify whether structural transpilation (ju2jmh) or generative AI (LLM) yields superior sensitivity and accuracy in regression detection.

CHAPTER 3

Research Methodology

This chapter presents a detailed description of the experimental procedures used to evaluate the effectiveness of automated microbenchmark generation. The work is organized into sequential phases, beginning with problem formalization and concluding with a high-precision execution environment.

3.1 Goal and Research Question

The primary objective of this empirical study is to systematically evaluate and compare the effectiveness of automated methodologies for generating Java Microbenchmark Harness (JMH) tests. This research investigates two distinct automated generation paradigms: a hybrid structural approach (ju2jmh-augmented LLM, where an LLM generates a JUnit test that is subsequently translated by a static tool) and a purely generative artificial intelligence approach (standalone LLM, where the benchmark is synthesized directly). The goal is to determine their practical viability in detecting real-world performance regressions across evolving software versions.
To guide this rigorous investigation, the experimental design is explicitly centered around a single Research Question:

RQ. *Between ju2jmh-augmented LLM and standalone LLM, which approach is more effective at detecting performance regressions?*

3.2 Context of the Application: Apache Ignite

Apache Ignite[1] is a highly complex, open-source distributed database, caching, and processing platform. Specifically, Apache Ignite functions as a distributed in-memory key-value data system utilized to process heavy database workloads, providing horizontal scaling across multiple nodes and supporting standard SQL interfaces. Selecting an appropriate study subject is a critical step in performance engineering research, as the chosen software must be large enough to exhibit real-world architectural complexities while maintaining a rich history of code evolution.
Apache Ignite was specifically chosen as the context of application for several fundamental reasons. First and foremost, its architectural nature makes it exceptionally relevant for strict performance evaluation. As a memory-centric distributed system designed for high-throughput and low-latency operations, its overall efficiency is highly sensitive to localized code changes. In such large-scale, interconnected environments, even seemingly minor performance degradations at the method level—such as inefficient memory allocations, suboptimal data serialization, or slight increases in execution time—can propagate rapidly through the system. This propagation can cause severe latency spikes or excessive resource consumption during distributed workloads. Consequently, identifying and mitigating these fine-grained performance regressions as early as possible is paramount to maintaining the system's overall reliability. Second, evaluating performance regressions in a distributed database system via traditional application-level macrobenchmarking is notoriously difficult. Deploying a full cluster of Apache Ignite to execute comprehensive, system-wide load tests for every single code commit is computationally expensive, highly complex to configure, and practically infeasible within agile Continuous Integration (CI) pipelines. This inherent difficulty makes Apache Ignite an ideal candidate to demonstrate the distinct value of microbenchmarking.

[1] Apache Ignite Github Repository

Furthermore, from a practical research perspective, the Apache Ignite repository provides a robust and necessary testbed for the experimental pipelines. To systematically evaluate the ju2jmh-augmented pipeline, the target project must possess a comprehensive and well-maintained suite of functional JUnit tests. Apache Ignite fulfills this requirement, providing the foundational unit tests that the ju2jmh static analysis tool translates into Java Microbenchmark Harness (JMH) scaffolding. Additionally, its extensive, publicly accessible version control history contains numerous real-world commit pairs—specifically, parent commits containing known performance regressions and their immediate bug-fixing child commits—which provides a clear ground truth for the experiment.
By conducting the experiments on Apache Ignite, this study ensures that the evaluation is grounded in realistic, industrial-scale software.

3.3 Experiment Configurations

The experimental configuration establishes a strict, reproducible pipeline designed to evaluate the microbenchmark generation methodologies across real-world software evolution.

3.3.1 Ju2jmh-augmented LLM

The ju2jmh-augmented LLM approach is a hybrid methodology developed to bridge the gap between purely generative AI and deterministic, rule-based automation. While Large Language Models (LLMs) like GPT-4 excel at understanding code intent and generating complex logic, they often struggle with the rigid boilerplate requirements and delicate configuration parameters of the Java Microbenchmark Harness (JMH)[2]. Conversely, the ju2jmh tool is highly effective at structurally transpiling existing JUnit tests into microbenchmarks but lacks the ability to "invent" new workloads or target specific methods that lack existing tests. As illustrated in Figure 3.1, this process involves executing each benchmark on two distinct versions of the software: a 'parent' commit containing the regression and a 'fix' commit where the issue is

[2]OpenJDK - Java Microbenchmark Harness (JMH) - Git repository

resolved. A benchmark is considered to have successfully detected the regression if it reports a statistically significant performance improvement in the fix commit relative to the parent commit.

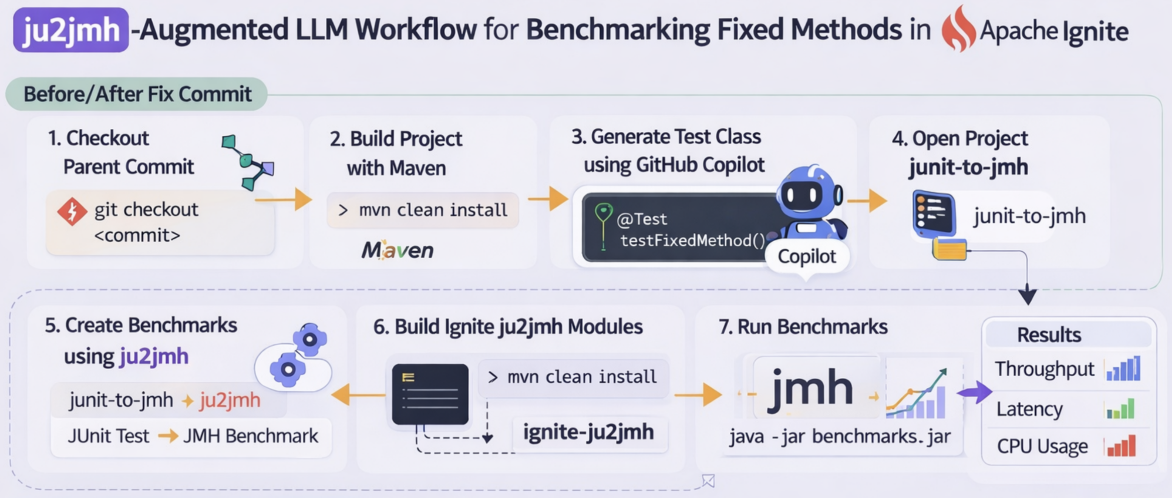

Figure 3.1: ju2jmh-augmented LLM works

The following structure details the execution of this methodology as applied to the Apache Ignite project, focusing on the isolation and detection of performance regressions.

Generative Workload Creation via GPT-4. The core of the "augmented" strategy. Instead of relying on existing unit tests—which might be too broad or functionally focused to detect a subtle performance lag—GitHub Copilot is used to generate a specific JUnit test class. The researcher directs the LLM to the "changed method" (the method targeted by the fix in the subsequent commit). The prompt instructions require the LLM to generate a JUnit test that stresses the performance of this specific logic. By using the LLM to write a *JUnit* test rather than a *JMH* benchmark directly, we bypass the LLM's tendency to misconfigure JMH annotations (like `@State` or `@Setup`). The resulting code is a high-coverage unit test designed to act as a performance workload.

I prompted GPT-4 to generate JUnit as follows: "Generate the class_name + Test class to test method_name of class_name"

And Copilot created a class for me in the following format:

Figure 3.2: JUnit Test class that by GPT-4

```
Here is a simple JUnit test class for the ceilPow2 method in IgniteUtils. It checks various edge cases and typical values.

package org.apache.ignite.internal.util;
import org.junit.Test;
import static org.junit.Assert.*;
import static org.apache.ignite.internal.util.IgniteUtils.ceilPow2;
public class IgniteUtilsTest {

@Test
public void testCeilPow2() {
// Edge cases
assertEquals(1, ceilPow2(0));
assertEquals(1, ceilPow2(1));
assertEquals(2, ceilPow2(2));
assertEquals(4, ceilPow2(3));
assertEquals(4, ceilPow2(4));
assertEquals(8, ceilPow2(5));
assertEquals(8, ceilPow2(8));
assertEquals(16, ceilPow2(9));
assertEquals(16, ceilPow2(15));
assertEquals(16, ceilPow2(16));
assertEquals(32, ceilPow2(17));
assertEquals(1024, ceilPow2(1000));
assertEquals(1024, ceilPow2(1024));
assertEquals(2048, ceilPow2(1025));

// Large value
assertEquals(1 << 30, ceilPow2((1 << 30) - 1));
assertEquals(1 << 30, ceilPow2(1 << 30));

// Test some more edge cases
assertEquals(2, ceilPow2(-1));  // We'll see what the actual behavior is

assertEquals(2, ceilPow2(-100)); // We'll see what the actual behavior is
}

}

```

Structural Transpilation of Benchmarks In this phase, the ju2jmh[3] tool is executed to transform the LLM-generated JUnit tests into JMH-compliant microbenchmarks. Specifically for the Apache Ignite project, this involves mapping the test logic into the Ignite-specific benchmarking modules(Figure 3.3). The tool scans the JUnit test for `@Test` annotations and method calls, then creates corresponding `@Benchmark`

[3]junit-to-jmh - Git repository

methods. It automatically handles the complex task of resource management, ensuring that Ignite nodes are started and stopped appropriately during the benchmark lifecycle—a task that LLMs frequently fail to handle correctly when generating pure JMH code from scratch.

Figure 3.3: ju2jmh generates Test Benchmarks

```
gradle converter:run
 ↪  --args="/Users/admiral/Documents/UNISA/Thesis/ignite/test/modules/core/src/test/java/
 ↪  /Users/admiral/Documents/UNISA/Thesis/ignite/test/modules/core/target/test-classes/
 ↪  /Users/admiral/Documents/UNISA/Thesis/ignite/test/modules/ju2jmh/src/jmh/java/
 ↪  org.apache.ignite.internal.util.IgniteUtilsTest"

```

Benchmark Execution and Data Collection The execution phase is conducted under strict environmental controls to minimize measurement noise and external influencing factors. Specifically, this isolated environment is configured by disabling Intel Turbo Boost, hyper-threading, and Address Space Layout Randomization (ASLR)[24].

Each benchmark is executed using a specific, prolonged JMH configuration: 10 forks, 3000 measurement iterations, 0 warm-up iterations, and a 100ms measurement time in sample mode. These exact parameters are not merely chosen to provide a high volume of data points; rather, they are adopted directly from the state-of-the-art methodology established by Traini et al [24].

Because the Java Virtual Machine (JVM) introduces severe performance fluctuations during dynamic Just-In-Time (JIT) compilation, developers often manually configure warm-up iterations to discard initial unstable measurements. However, definitively demonstrated that Java microbenchmarks do not always reliably reach a steady state, and manual estimates of the warm-up phase by developers are frequently inaccurate, leading to distorted performance assessments[24].

By setting warm-up iterations to 0 and running a massive amount of measurement iterations (3000) across 10 independent JVM forks(Figure 3.4), this configuration completely captures the raw execution times of the entire benchmark lifecycle. This

completely bypasses potentially inaccurate, predefined warm-up estimates and allows the subsequent statistical analysis to accurately evaluate the true performance distribution [24].

Figure 3.4: Run the Benchmark

```
java -jar target/jmh-benchmarks.jar
↪  org.apache.ignite.internal.util.IgniteUtilsTest.ceilPow2 -f 10 -i 3000 -r 100ms -wi 0
↪  -rff benchmark-results.json -rf json -bm avgt -tu ms
```

Figure 3.5: Results of Benchmark Execution


```
[
    {
        "jmhVersion" : "1.37",
        "benchmark" : "org.apache.ignite.internal.util.IgniteUtilsTest._Benchmark.benchma
        ↪  rk_testCeilPow2ForMaxInt",
        "mode" : "ss",
        "threads" : 1,
        "forks" : 10,
        "jvm" : "/usr/lib/jvm/java-8-openjdk-amd64/jre/bin/java",
        "jvmArgs" : [
            "-Xms8G",
            "-Xmx8G"
        ],
        "jdkVersion" : "1.8.0_462",
        "vmName" : "OpenJDK 64-Bit Server VM",
        "vmVersion" : "25.462-b08",
        "warmupIterations" : 0,
        "warmupTime" : "single-shot",
        "warmupBatchSize" : 1,
        "measurementIterations" : 3000,
        "measurementTime" : "single-shot",
        "measurementBatchSize" : 1,
        "primaryMetric" : {
            "score" : 0.03937091946666684,
            "scoreError" : 0.021704992844603088,
            "scoreConfidence" : [
                0.017665926622063753,
                0.06107591231126993
            ],
            "scorePercentiles" : {
                "0.0" : 0.002597,
                "50.0" : 0.009386,
                "90.0" : 0.04442780000000003,
                ...
                "99.99" : 67.36613945558948,
                "99.999" : 75.15173,
                "99.9999" : 75.15173,
                "100.0" : 75.15173
```

```
            },
            "scoreUnit" : "ms/op",
            "rawData" : [
                [
                    65.150917,
                    0.118482,
                    0.075426,
                    0.082394,
                    0.094064,
                    0.074934,
                    0.066965,
                    0.069877,
                    0.160059,
                    0.136896,
                    0.138922,
                    0.180365,
                    0.14692,
                    0.131016,
                    ...
                                        0.009721,
                    0.006583,
                    0.01044
                ]
            ]
        },
        "secondaryMetrics" : {
        }
    }
]
```


Iterative Verification of the Fixing Commit The final step is the repetition of the entire process for the "after-fix" commit. The researcher switches to the commit where the regression was resolved and repeats the build, transpilation, and execution steps. Crucially, the *exact same* LLM-generated workload is used for both versions. This ensures that any observed difference in execution time is purely the result of the code changes in the software under test, rather than variations in the test logic itself. This comparative loop concludes the methodology, providing the raw data needed to determine if the ju2jmh-augmented LLM approach is superior to traditional methods in detecting real-world performance regressions.

Through this structured process, the methodology ensures that the "creativity" of the LLM is harnessed within the "safety" of a deterministic tool, resulting in benchmarks that are both relevant to the code changes and statistically rigorous.

3.3.2 Standalone LLM

The standalone LLM approach serves as a critical comparative baseline in this research, representing a "pure" generative methodology for performance test creation. Unlike the hybrid approach described in the previous section, the standalone method evaluates the ability of Large Language Models (LLMs)—specifically GPT-4 [4] through Copilot IDE Plugin — to act as autonomous performance engineers. In this paradigm, the AI is not used to generate a simple unit test for later conversion; rather, it is tasked with the direct synthesis of production-ready Java Microbenchmark Harness (JMH) code. This section details the five-step experimental pipeline used to implement this methodology within the Apache Ignite project.

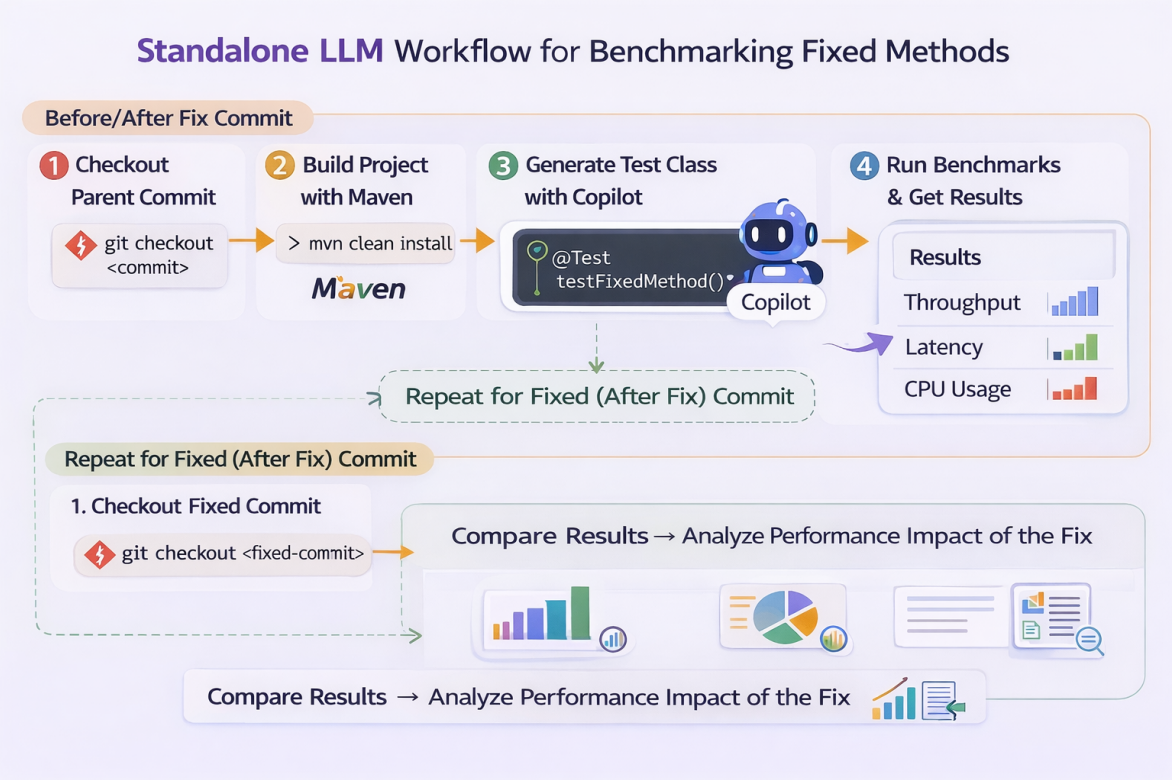

Figure 3.6: Standalone LLM works

Generative Workload Creation via GPT-4 The most distinctive phase of this methodology is the generative process. The researcher directs GPT-4 to the specific method that underwent a performance-related change in the subsequent "fixing" commit. The LLM is then prompted to generate a standalone JMH benchmark class

[4] OpenAI Website

from scratch. Unlike generating a standard JUnit test, this requires the AI to handle complex performance-specific concerns, including the correct application of JMH annotations such as `@Benchmark`, `@State`, and `@Setup`.

The LLM must also manage the lifecycle of the Apache Ignite environment within the benchmark. For instance, it must often instantiate an Ignite node or a cache instance before the measurement iterations begin and ensure a proper shutdown afterward. This phase tests whether the AI can avoid common microbenchmarking pitfalls, such as dead-code elimination or constant folding, without the assistance of a deterministic transpilation tool like ju2jmh.

I prompted GPT-4 to generate JUnit as follows: "Generate a benchmark class of *class_name* to benchmark *method*"

And GPT-4 created a class for me in the following format:

Figure 3.7: JUnit Test class that by GPT-4

```
Here is a simple JUnit test class for the ceilPow2 method in IgniteUtils. It checks
↪ various edge cases and typical values.

package org.apache.ignite.internal.util;
import org.junit.Test;
import static org.junit.Assert.*;
import static org.apache.ignite.internal.util.IgniteUtils.ceilPow2;
public class IgniteUtilsTest {

@Test
public void testCeilPow2() {
// Edge cases
assertEquals(8, ceilPow2(8));
assertEquals(16, ceilPow2(9));
assertEquals(16, ceilPow2(15));
assertEquals(16, ceilPow2(16));
assertEquals(32, ceilPow2(17));
assertEquals(1024, ceilPow2(1000));
assertEquals(1024, ceilPow2(1024));

// Large value
assertEquals(1 << 30, ceilPow2((1 << 30) - 1));
assertEquals(1 << 30, ceilPow2(1 << 30));

// Test some more edge cases
assertEquals(2, ceilPow2(-1));  // We'll see what the actual behavior is

assertEquals(2, ceilPow2(-100)); // We'll see what the actual behavior is
}
}
```

Environment Transition to junit-to-jmh After the generated benchmark is compiled and packaged into the JMH runner JAR, the execution phase is initiated. To ensure that the results are statistically rigorous and comparable to the augmented approach, the standalone benchmarks are run using a highly specific configuration adopted from the state-of-the-art methodology established by Traini et al. Each benchmark is executed with 10 forks to capture inter-JVM variance and 3000 measurement iterations to provide a robust dataset for analysis. These massive parameters are necessary to completely capture the true execution distributions across independent JVM instantiations, ensuring the high statistical significance required to rigorously study JVM performance[6].
The execution command is as follows:

```
java -jar target/jmh-benchmarks.jar [BenchmarkName] -f 10 -i 3000 -r 100ms -wi 0 -bm
↪  sample -tu ms -rf json -rff benchmark-results.json
```

Figure 3.8: The execution command

By setting the benchmarking mode to "sample" and the iteration time to 100ms, the experiment collects granular latency data for every single invocation. Disabling warm-up iterations (-wi 0) ensures that we capture the raw performance profile of the code from the very beginning of its lifecycle. As demonstrated by Traini et al., Java microbenchmarks do not always reliably reach a steady state, and manual estimates of the warm-up phase are frequently inaccurate, leading to distorted performance assessments [24]. Therefore, explicitly bypassing predefined warm-up estimates is essential, as it allows the subsequent hierarchical bootstrap analysis to accurately evaluate the true performance distribution and reliably detect regressions.

Comparative Evaluation of the Fixed Commit The final stage is the repetition of the process for the "after-fix" commit. The researcher switches to the version of the code where the regression was resolved and rebuilds the project. Crucially, the *exact same* benchmark code generated by the LLM in Step 3 is applied to this new version. This "controlled workload" approach ensures that any shift in execution time is strictly attributable to the code optimization in Apache Ignite rather than variations in the

test logic. By comparing the results from the parent and fixed commits, the study determines if the standalone LLM was successful in generating a benchmark with sufficient sensitivity to distinguish between the two performance states. This five-step process provides a clear measurement of the LLM's effectiveness in automating the detection of real-world software regressions.

CHAPTER 4

Empirical Study

In this chapter, the results from the statistical analysis are showed and are discussed to compare the approaches in a quantitive way, answering the Research Question

4.1 System Overview

The primary goal of this chapter is to present the empirical results obtained from our experimental evaluation and to rigorously compare the effectiveness of two distinct automated methodologies for generating Java Microbenchmark Harness (JMH) tests. Specifically, this analysis focuses on contrasting a hybrid, deterministic translation approach (ju2jmh-augmented LLM) against a purely generative artificial intelligence approach (standalone LLM).

First, we examine the overall *Detection Rate* to determine the proportion of generated microbenchmarks that successfully identify a statistically significant performance degradation in the expected direction. Second, we assess the *Statistical Reliability* of these findings, utilizing confidence intervals to ensure that the detected differences are true reflections of code execution rather than environmental noise. Finally, we analyze the *Effect Size* to quantify the practical magnitude of the performance changes captured by each methodology. By structuring the analysis around these metrics, this

chapter provides a definitive, data-backed assessment of which generation paradigm yields the most sensitive and accurate performance tests[1].

4.2 Experiment configuration

All generated Java Microbenchmark Harness (JMH) tests—from both the ju2jmh-augmented pipeline and the standalone LLM pipeline—were executed on a dedicated laboratory machine. The hardware specifications of the machine include an Intel Core i9-10980XE processor running at 3.0 GHz and 62.5 GB of RAM. The machine operates on Ubuntu 24, providing a modern, stable Linux environment for continuous performance assessment.
JMH Execution Parameters To capture a statistically robust distribution of execution times, we bypassed standard default JMH configurations in favor of a highly granular sampling strategy. For each microbenchmark across both the before-fix and after-fix commits, the following strict JMH parameters were utilized:

- Forks (-f 10): The benchmark was executed across 10 entirely separate JVM instances to account for inter-fork variance and JVM initialization non-determinism
- Measurement Iterations (-i 3000): A massive sample size of 3,000 iterations was collected per fork to ensure statistical significance
- Warm-up Iterations (-wi 0): Warm-up iterations were explicitly set to 0. This allows the measurement to capture the raw execution distribution, which is then handled by our hierarchical bootstrap analysis
- Benchmark Mode: The framework was set to single shot benchmark mode
- Measurement Metric: The tests were configured to record the average time measurement to assess the execution duration of the target methods.

Furthermore, to guarantee robust data collection, this exact execution cycle was repeated 10 times for each microbenchmark, generating distinct JSON output files (e.g.,

[1] Results - Git repository

benchmark-results_0.json through benchmark-results_9.json) for the hierarchical bootstrap script to process.

System-Level Isolations (Foresights) Even on a dedicated machine, default OS and CPU behaviors can introduce unacceptable levels of measurement bias. To prevent the system from influencing the executions, the following "foresights" (system-level tuning protocols) were applied prior to running the benchmarks, aligning with established best practices in performance engineering recommended by Traini et al. [24]:

- Disable Intel Turbo Boost: Dynamic CPU frequency scaling was turned off to ensure the processor ran at a constant, predictable clock speed.
- Disable Hyper-Threading: Simultaneous multithreading was disabled to eliminate unpredictable core-sharing contention between threads.
- Disable ASLR: Address Space Layout Randomization was deactivated to guarantee memory placement consistency across different JVM forks.
- Fix Heap Memory: The total amount of JVM heap memory was explicitly fixed to 8 GB using the -Xmx flag, standardizing the overhead and frequency of Garbage Collection pauses.
- Disable Background Processes: Any Unix daemon not strictly mandatory for the microbenchmarks' executions was disabled to free up CPU cycles and eliminate background OS interference.

By strictly adhering to this configuration, we ensured that the collected execution times were strictly representative of the code's inherent performance, providing a highly reliable foundation for the subsequent statistical analysis

4.3 Evaluation Metrics

Statistical Significance via Hierarchical Bootstrapping Java Microbenchmark Harness (JMH) data possesses an inherent two-level hierarchical structure: variation occurs both between different JVM instances (forks) and within the measurement

iterations of a single fork. Standard statistical methods that ignore this hierarchy assume all measurements are independent, frequently leading to overconfident or false conclusions [7].

To rigorously assess whether a benchmark detected a regression, we utilize a non-parametric hierarchical bootstrap method that accurately models this dual-level uncertainty. The process performs 10,000 bootstrap iterations. In each iteration, it randomly resamples the forks with replacement, and subsequently resamples the iterations within those selected forks with replacement [2, 4]. For every bootstrap sample, the algorithm calculates the ratio of the sample means between the parent commit (before-fix) and the child commit (after-fix).

From these 10,000 resampled ratios, a 95% Confidence Interval (CI) is constructed. The benchmark's ability to detect a performance change is evaluated using this interval:

- If the 95% CI strictly excludes the value 1.0, the performance difference between the two software versions is deemed statistically significant [7].
- If the entire 95% CI is greater than 1.0 (e.g., [1.465, 1.531]), it demonstrates with 95% confidence that the before-fix version executed significantly slower than the after-fix version, indicating a successfully detected regression.

Non-Parametric Effect Size While statistical significance confirms that a performance difference exists and is not merely random noise, the effect size quantifies the actual magnitude and practical relevance of that difference. Given the non-normal distribution of performance data, we utilize two standard non-parametric effect size metrics:

- Vargha-Delaney A_{12} : This metric measures the probability that a randomly selected execution time from the before-fix version is slower than a randomly selected execution time from the after-fix version. An A_{12} value of 0.5 indicates complete overlap (no effect), while values moving toward 1.0 represent increasingly large regression effects (e.g., >0.71 is considered a large effect) [24, 27].

- Cliff's δ: As a complementary measure, Cliff's δ quantifies the degree of dominance of one distribution over another. The magnitude is assessed using established thresholds, where absolute values below 0.147 are considered negligible, and values above 0.474 are considered large[9, 13]

Detection Rate To directly answer the core Research Question—Between ju2jmh-augmented LLM and standalone LLM, which approach is more effective at detecting performance regressions?—we aggregate the statistical findings to calculate the overall Detection Rate for each generation pipeline. The detection rate is defined as the percentage of generated microbenchmarks that successfully yield a statistically significant, positive regression detection (where the 95% CI lower bound is > 1.0). By comparing the detection rates of the ju2jmh-augmented LLM approach against the standalone LLM approach, we can definitively determine which methodology produces the most sensitive and reliable performance tests.

4.4 RQ: Between ju2jmh-augmented LLM and standalone LLM, which approach is more effective at detecting performance regressions?

Based on the aggregated statistical outputs across the evaluated Apache Ignite dataset, the empirical evidence provides a clear and definitive answer: the Standalone LLM approach is demonstrably superior and more effective at detecting actual performance regressions than the ju2jmh-augmented approach. To fully substantiate this conclusion, the following subsections break down the generation outputs, beginning with a high-level summary and proceeding into granular, pair-by-pair data analyses. As demonstrated in Table 4.1, the ju2jmh-augmented pipeline generated a total of 75 benchmarks across the commit pairs. However, the hierarchical bootstrap analysis revealed that zero (0.0%) of the ju2jmh benchmarks were able to identify a statistically significant performance regression. The structural translation of the functional JUnit tests failed to isolate the performance-critical paths affected by the code changes.

Table 4.1 summarizes the overarching performance of both generation methodologies across the entirety of the evaluated dataset.

Table 4.1: Overall Regression Detection Summary

Generation Methodology	Total Benchmarks Generated	Statistically Significant Differences	Confirmed Regressions Detected	Unexpected Results	Overall Detection Rate
ju2jmh-augmented LLM	75	0	0	0	0.0%
Standalone LLM	53	6	2	4	3.8%

In stark contrast, out of the 53 total microbenchmarks generated natively by the Standalone LLM, the statistical analysis confirmed 6 statistically significant performance differences. Crucially, 2 of these were successful, confirmed regression detections where the benchmark isolated the exact performance degradation caused by the parent commit, yielding an overall detection rate of 3.8%. The LLM also produced 4 "unexpected results" (instances where the pre-fix code executed significantly faster, indicating potential noise or misaligned test workloads). Ultimately, the generative LLM approach was responsible for 100% of the successful regression detections.
To contextualize where and why the LLM outperformed the deterministic tool, Table 4.2 breaks down the benchmark generation outputs by the specific Apache Ignite commit pairs evaluated.

Commit Pair Context	Methodology	Generated Benchmarks	Regressions Detected	Unexpected Results
Pair 1 (OdbcRequestHandler)	ju2jmh+LLM	7	0	0
Pair 1 (OdbcRequestHandler)	Standalone LLM	0	0	0
Pair 2 (IGFS & Hadoop FS)	ju2jmh+LLM	68	0	0
Pair 2 (IGFS & Hadoop FS)	Standalone LLM	51	2	4
Pair 3 (IgniteUtils)	ju2jmh+LLM	0	0	0
Pair 3 (IgniteUtils)	Standalone LLM	2	0	0

Table 4.2: Statistical Detection Breakdown by Architectural Component

The detailed breakdown reveals that the LLM's successful detections were isolated within **Pair 2**, which targeted the complex distributed I/O operations of the IGFS (Ignite In-Memory File System) and Hadoop File System suites. Unconstrained by intermediate functional tests, the LLM successfully synthesized two highly sensitive benchmarks that captured the regressions perfectly. Table 4.3 highlights the specific statistical metrics of these successful detections.

Table 4.3: Detailed Insights into Successful LLM Detections

Benchmark Name (Standalone LLM)	95 % Confidence Interval	Measured Slowdown (Pre-Fix vs Post-Fix)	Verdict
benchmarkReadExternal	Strictly > 1.0	10.74% ± 8.56% slower	**DETECTED**
benchmarkWriteExternal	Strictly > 1.0	8.48% ± 8.08% slower	**DETECTED**

For this exact same commit pair (Pair 2), the ju2jmh pipeline generated 68 distinct benchmarks, yet all produced tiny, non-significant deltas (ranging from approximately -5% to +6%) with a Cliff's δ near 0, failing entirely to trigger a single detection. This indicates that while both methods executed on the same code, the LLM natively generated a much more representative and stressful payload for the file system operations.

👉 **Answer to RQ.** Standalone LLM as the primary, most effective vehicle for isolating and definitively detecting software performance regressions. These results, validated through rigorous hierarchical bootstrap analysis, establish Standalone LLM as a high-sensitivity testing vehicle, though its flexibility should be ideally balanced with deterministic structural safety to ensure optimal benchmark calibration.

4.5 Discussion

The empirical findings of this study establish that the standalone Large Language Model (LLM) is demonstrably more effective at detecting performance regressions than the ju2jmh-augmented approach. However, beyond the raw detection rates, a deeper evaluation of the execution data highlights fundamental trade-offs between utilizing static analysis tools applied to functional tests versus natively generating performance tests via artificial intelligence.

The Limitations of Rule-Based Translation While the ju2jmh tool offers high maturity, excellent traceability, and a reliable baseline of structural safety, it remains

fundamentally constrained by the logic of the underlying JUnit tests. Functional unit tests are traditionally designed to verify logical correctness; they rarely invoke the appropriate workload scale, data volume, or complex state management required to accurately stress performance-critical paths. Because ju2jmh deterministically loops this existing functional logic without an awareness of performance needs, it frequently yields execution states incapable of isolating efficiency degradations. Within our evaluation, this structural rigidity ultimately led to a 0.0% regression detection rate.

The Power and Pitfalls of Generative AI Conversely, the Standalone LLM approach bypasses the limitations of intermediate functional tests entirely. By utilizing its vast semantic understanding of the Java Microbenchmark Harness (JMH) framework, the LLM natively constructs more idiomatic, performance-focused workloads directly from the target source code. This generative flexibility allowed it to successfully isolate critical execution paths and achieve the only statistically significant, verified regression detections in the study. Nevertheless, this purely generative approach is not without its vulnerabilities. As evidenced by the presence of unexpected results and occasional massive, hallucinated execution scales in certain commit pairs, unguided LLMs remain susceptible to miscalibration. The AI can sometimes invent non-comparable states or unrealistic data structures that yield statistically significant, yet practically invalid, performance differences.

Practical Implications and Recommendations These findings suggest that neither methodology is flawlessly sufficient in isolation. Instead, best practices in modern automated performance engineering dictate a **dual-pipeline strategy**. Developers should utilize the ju2jmh pipeline to establish safe, structurally sound baseline reporting and to perform face-validity checks, ensuring that the benchmark logic strictly adheres to functional expectations without manual effort. Concurrently, development teams must rely on the Standalone LLM as the primary, most effective vehicle for actually stressing the code, isolating performance anomalies, and definitively detecting software performance regressions. By combining the structural safety of static analysis with the generative power of AI, developers can achieve a more robust and scalable continuous performance assessment pipeline.

CHAPTER 5

Threats to Validity

This chapter describes the various aspects that could potentially threaten the validity of the empirical study conducted in this thesis. To systematically address these concerns and ensure the robustness of the experimental design, the potential threats are categorized into four main areas: construct validity, internal validity, external validity, and conclusion validity.

5.1 Construct Validity

The primary threat involves how software performance regressions are captured and quantified. Relying on simple arithmetic averages of execution times is inherently flawed and could misrepresent actual performance. To mitigate this threat, the execution data was collected using the industry-standard Java Microbenchmark Harness (JMH), which is specifically designed to extract fine-grained distributions of execution times while preventing aggressive Java Virtual Machine (JVM) optimizations, such as dead-code elimination and constant folding. Furthermore, rather than evaluating raw time differences, the study quantified performance changes using robust effect size metrics to accurately capture the true, practical impact of the software evolution[1][2][3].

5.2 Internal Validity

For performance benchmarking, the most significant threat is the extreme non-determinism of the JVM—such as Just-In-Time (JIT) compilation and Garbage Collection—as well as operating system noise. To mitigate this, all microbenchmarks were executed on a strictly isolated laboratory bare-metal machine. Hardware and OS features known to introduce variability, including Intel Turbo Boost, hyper-threading, and Address Space Layout Randomization (ASLR), were explicitly disabled. The JVM heap memory was fixed to 8GB, and background Unix daemons were turned off. Additionally, the JMH setup was rigorously configured to use 10 forks and 3000 measurement iterations to bypass JVM warmup instabilities. The configuration applied, together with the setup of the execution environment for the microbenchmarks, aims to remove most variability coherently with the state-of-the-art methodology established by Traini et al. Another internal threat involves the inherent randomness of Large Language Models (LLMs) and variations in the generated code. To prevent this from skewing the comparison, the exact same generated benchmark source files were ported directly from the pre-fix (parent) commit to the post-fix (child) commit, guaranteeing that the testing payload remained strictly identical across the two evaluated software versions[[24]].

5.3 External Validity

External validity concerns the generalizability of the obtained results beyond the specific conditions under which the study was conducted. A primary threat to this study is that the evaluation is confined to three specific commit pairs within a single open-source project (Apache Ignite). Moreover, the experiment evaluated only one specific static analysis tool (ju2jmh) and one generative AI approach (the standalone LLM). To mitigate the single-project limitation, Apache Ignite was deliberately chosen because it is a highly complex, industrial-grade distributed database that perfectly represents real-world architectural intricacies. Furthermore, the three selected commit pairs were carefully filtered to span completely distinct operational domains: external query processing, distributed file system I/O, and core algorithmic utilities. While

these steps ensure a diverse and realistic evaluation, future replications of this study on a larger number of varied Java projects and utilizing different LLMs would be necessary to fully corroborate our findings[33][35].

5.4 Conclusion Validity

Conclusion validity concerns the extent to which the relationship between the experimental treatment—in this study, the benchmark generation methodologies—and the observed outcome is statistically sound. To ensure that the identified performance shifts represent genuine regressions rather than environmental artifacts, we mitigate this threat by employing a rigorous statistical framework comprising hierarchical bootstrap analysis and non-parametric effect size metrics. Consequently, any results that fail to meet the established criteria for statistical significance are excluded from the final set of confirmed detections; while such cases may be analyzed for diagnostic context, they are not considered valid evidence of a performance regression.

CHAPTER 6

Conclusion

Evaluations conducted on actual performance regressions within the sophisticated Apache Ignite ecosystem revealed that the standalone LLM approach is more effective. While the rule-based conversion method was unable to pinpoint confirmed regressions, the LLM successfully detected several issues. These conclusions were validated through a rigorous hierarchical statistical framework that accounts for the inherent variability in performance measurements.

Despite its superior detection capabilities, the LLM occasionally produced miscalibrated benchmarks. To address this, the study proposes an integrated strategy that utilizes the structural reliability of deterministic tools alongside the creative versatilitiy of generative models.

Future research should extend this evaluation to a wider variety of open-source ecosystems and test the performance of emerging state-of-the-art models. A primary focus should be the development of automated validation loops to correct benchmarking errors and prevent LLM miscalibration before the execution phase. Finally, implementing optimization strategies, such as batching benchmarks or dynamically adjusting execution parameters, will be vital for making LLM-driven performance testing both scalable and cost-effective for real-world development environments.

Bibliography

[1] M. Abdullah, D. G. Reichelt, V. Horký, L. Bulej, T. Bureš, and P. Tůma, "A combined approach to performance regression testing resource usage reduction," in *Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering*, ser. PROMISE '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 75–84. [Online]. Available: https://doi.org/10.1145/3727582.3728690 (Citato alle pagine 1, 3, 7 e 8)

[2] L. Traini, F. Di Menna, and V. Cortellessa, "Ai-driven java performance testing: Balancing result quality with testing time," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 443–454. [Online]. Available: https://doi.org/10.1145/3691620.3695017 (Citato alle pagine 1, 3, 7, 8, 9, 21 e 38)

[3] S. He, I. K. Kim, and W. Wang, "A study of java microbenchmark tail latencies," in *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 77–81. [Online]. Available: https://doi.org/10.1145/3578245.3584690 (Citato alle pagine 2, 8 e 9)

[4] M. Jangali, Y. Tang, N. Alexandersson, P. Leitner, J. Yang, and W. Shang, "Automated generation and evaluation of jmh microbenchmark suites from unit

tests," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1704–1725, 2023. (Citato alle pagine 2, 3, 8, 9, 17, 18 e 38)

[5] M. Grambow, D. Kovalev, C. Laaber, P. Leitner, and D. Bermbach, "Using microbenchmark suites to detect application performance changes," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2575–2590, 2023. (Citato alle pagine 2, 8 e 21)

[6] S. M. Blackburn, Z. Cai, R. Chen, X. Yang, J. Zhang, and J. Zigman, "Rethinking java performance analysis," ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 940–954. [Online]. Available: https://doi.org/10.1145/3669940.3707217 (Citato a pagina 3)

[7] D. Campos, L. Martins, E. Guglielmi, M. Tucci, D. D. Pompeo, S. Scalabrino, V. Cortellessa, D. D. Nucci, and R. Oliveto, "Identifying and replicating code patterns driving performance regressions in software systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.05851 (Citato alle pagine 7, 8, 14, 15, 16 e 38)

[8] H. Zhang, X. Jiang, S. Li, J. Chen, and D. Wang, "Jmh-prophet: Optimizing java microbenchmark performance predictions with multimodal deep learning techniques," in *2025 25th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*, 2025, pp. 248–257. (Citato a pagina 7)

[9] D. Costa, C.-P. Bezemer, P. Leitner, and A. Andrzejak, "What's wrong with my benchmark results? studying bad practices in jmh benchmarks," *IEEE Transactions on Software Engineering*, vol. 47, no. 7, pp. 1452–1467, 2021. (Citato alle pagine 7, 9, 12, 21 e 39)

[10] L. Liao, S. Eismann, H. Li, C.-P. Bezemer, D. E. Costa, A. van Hoorn, and W. Shang, "Early detection of performance regressions by bridging local performance data and architectural models," 2024. [Online]. Available: https://arxiv.org/abs/2408.08148 (Citato alle pagine 7 e 8)

[11] N. Japke, C. Witzko, M. Grambow, and D. Bermbach, "The early microbenchmark catches the bug – studying performance issues using micro-

and application benchmarks," in *Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing*, ser. UCC '23. ACM, Dec. 2023, p. 1–10. [Online]. Available: http://dx.doi.org/10.1145/3603166.3632128 (Citato alle pagine 8 e 9)

[12] S. He, I. K. Kim, and W. Wang, "A study of java microbenchmark tail latencies," in *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 77–81. [Online]. Available: https://doi.org/10.1145/3578245.3584690 (Citato a pagina 8)

[13] N. Japke, M. Grambow, C. Laaber, and D. Bermbach, "µoptime: Statically reducing the execution time of microbenchmark suites using stability metrics," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 8, p. 1–26, Oct. 2025. [Online]. Available: http://dx.doi.org/10.1145/3715322 (Citato alle pagine 8, 9 e 39)

[14] C. Laaber and P. Leitner, "An evaluation of open-source software microbenchmark suites for continuous performance assessment," in *2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR)*, 2018, pp. 119–130. (Citato alle pagine 8 e 14)

[15] H. Samoaa and P. Leitner, "An exploratory study of the impact of parameterization on jmh measurement results in open-source projects," in *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 213–224. [Online]. Available: https://doi.org/10.1145/3427921.3450243 (Citato alle pagine 8, 9 e 12)

[16] A. Trovato, L. Traini, F. Di Menna, and D. Di Nucci, "Amber: Ai-enabled java microbenchmark harness," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2025, pp. 762–766. (Citato alle pagine 9 e 20)

[17] D. Beyer, S. Löwe, and P. Wendler, "Reliable benchmarking: requirements and solutions," *International Journal on Software Tools for Technology Transfer*,

vol. 21, no. 1, pp. 1–29, 2019. [Online]. Available: https://doi.org/10.1007/s10009-017-0469-y (Citato a pagina 11)

[18] W. Zhong, C. Li, K. Liu, J. Ge, B. Luo, T. F. Bissyandé, and V. Ng, "Benchmarking and categorizing the performance of neural program repair systems for java," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 1, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3688834 (Citato alle pagine 13, 14, 21 e 22)

[19] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024. (Citato alle pagine 14, 16 e 21)

[20] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," 2023. [Online]. Available: https://arxiv.org/abs/2302.11382 (Citato a pagina 15)

[21] L. A. Loss and P. Dhuvad, "An llm-based genetic algorithm for prompt engineering," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '25 Companion. New York, NY, USA: Association for Computing Machinery, 2025, p. 527–530. [Online]. Available: https://doi.org/10.1145/3712255.3726633 (Citato a pagina 16)

[22] M. Jangali, K. Yao, Y. Tang, D. E. Costa, and W. Shang, "Batch execution of microbenchmarks for efficient performance testing," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2025, pp. 603–607. (Citato a pagina 17)

[23] M. Rodriguez-Cancio, B. Combemale, and B. Baudry, "Automatic microbenchmark generation to prevent dead code elimination and constant folding," in *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 132–143. (Citato a pagina 20)

[24] L. Traini, V. Cortellessa, and D. Di Pompeo, "Towards effective assessment of steady state performance in java software: are we there yet?" *Empir Software Eng*, vol. 28, no. 13, 2023. (Citato alle pagine 21, 28, 29, 33, 37, 38 e 45)

[25] Z. Ding, J. Chen, and W. Shang, "Towards the use of the readily available tests from the release pipeline as performance tests. are we there yet?" in *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 1435–1446. (Citato a pagina 21)

[26] C. Zhi, Z. Hu, D. Li, C. Qin, M. Li, Z. Feng, and X. Zhao, "Chatperftest: A framework for llm-based jmh microbenchmark generation," *SSRN Electronic Journal*, 2025, preprint. [Online]. Available: https://ssrn.com/abstract=5081001 (Citato a pagina 22)

[27] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000. (Citato a pagina 38)

Acknowledgments